

Vaan Megam Networks

Task: TDMA Schedule Planner and Optimizer



NAME: SUDARSHAN V

ROLL NO: 2023105063

COLLEGE OF ENGINEERING GUINDY - ANNA
UNIVERSITY, CHENNAI

1. What I built

The task was to plan a collision-free TDMA schedule for 16 radios and then try it in an EMANE emulation. I did it in two parts. The first part is a Python program, which I call the Brain. It reads the coordinates of the radios and decides which radio may transmit in which time slot. The second part is a bridge and a test setup that hands that schedule to EMANE and checks what happens to real packets.

For the 16-node demo grid (300 m spacing, 500 m range) the Brain needs 9 slots, and 9 is proven to be the minimum. Giving every radio its own slot would take 16, so the frame is about 1.78 times more efficient. On 150 random 16-node layouts the best possible frame averages 7.77 slots, and my program reached the proven best in all 150. The independent checker found no conflicts and all 77 tests pass.

In EMANE, all the radios I checked accepted the schedule. Two radios that cannot hear each other but both talk to a third one lost 6 to 14 percent of their packets with my schedule, and 30 percent when I deliberately put them in the same slot. That points the right way, but it is a small test and I explain its limits in section 6.

2. How I thought about the problem

I treated each radio as a point in a graph and drew a link between two radios if they are within 500 m of each other. Two radios must not share a slot in two cases. The first is when they are directly linked. The second is the hidden terminal case, where A and C cannot hear each other but both reach B, so their signals clash at B.

Putting both rules together, two radios clash exactly when they are one or two hops apart. If I draw that as a new graph, the scheduling job becomes ordinary graph colouring. Each colour is a slot, and the number of colours is the length of the frame. Radios three or more hops apart may share a colour, and that sharing is the spatial reuse the task asks for.

Finding the smallest number of colours is NP-hard, so I did not try to be clever about the exact answer at first. I used a good heuristic and then checked how close it was to a lower bound. A radio and all of its neighbours are always within two hops of each other, so they all need different slots. The largest such group gives a lower bound on the frame length. If my answer equals the bound, it is the best possible.

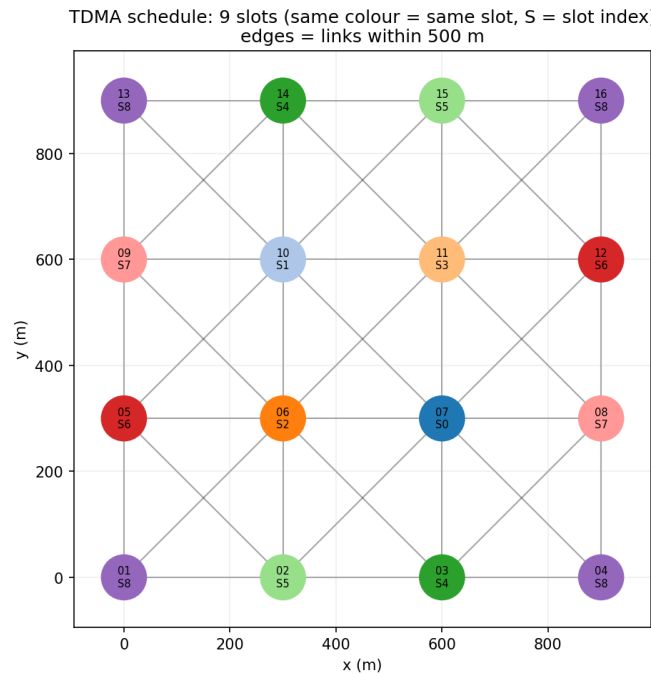
3. The algorithm

- DSATUR colouring: it always colours the radio with the most restricted choices first, which finds trouble early. I picked it over plain greedy because of that.
- 300 random restarts with a fixed seed of 42, so the result is the same every run.
- A small local search that tries to empty the highest slot by moving its radios into free lower slots.
- An exact search with a 5 second limit. If the heuristic result is above the lower bound, it asks whether one slot fewer is possible. If the answer is no, the result is proven optimal.
- An independent checker that recomputes every one-hop and two-hop pair from the raw coordinates, with separate code from the scheduler. A schedule is only printed if it passes.

I also looked at integer programming and simulated annealing. I left them out because the exact search already proves the result at this size, and they would only add code.

4. Results of Part 1

The schedule for the demo grid is below. The four corner radios (01, 04, 13 and 16) share slot 8 even though they are 900 m apart.



Slot	Radios that transmit
0	07
1	10
2	06
3	11
4	03, 14
5	02, 15
6	05, 12
7	08, 09
8	01, 04, 13, 16

Changing the radio range on the same grid changes the frame length, as expected. More range means more neighbours and fewer chances to reuse a slot.

Range (m)	Links	Slots	Lower bound	Optimal
250	0	1	1	yes
350	24	5	5	yes
500	42	9	9	yes
650	58	12	12	yes
850	90	16	16	yes

On 150 random layouts the plain greedy method matched the best answer 147 times, DSATUR 149 times, and the full pipeline with the exact step 150 times. So the heuristics are already good at this size, and the exact step is what turns good into proven. A larger test with 80 random radios in a 3 km square gave 13 slots, equal to the lower bound, in about half a second.

4.1 A second check in MATLAB

I did not want to trust only my own Python code, so I wrote the same scheduler again in MATLAB. It uses DSATUR with 300 restarts, needs no toolboxes and also runs in Octave. The script finds the colouring, builds the Slot by Node matrix, re-checks every one-hop and two-hop pair from the coordinates, and simulates one frame of packets. It found 9 slots with a lower bound of 9, so this answer is also the best possible, and the checker reported 0 conflicts for both the MATLAB schedule and the Python schedule.

The MATLAB schedule is not identical to the Python one. Node_07 is in slot 0 in both, but Node_02 is in slot 7 in MATLAB and slot 5 in Python. Random tie-breaking simply lands on a different colouring, and both are valid. Figure 1 shows the MATLAB schedule on the grid and Figure 2 shows it as a matrix.

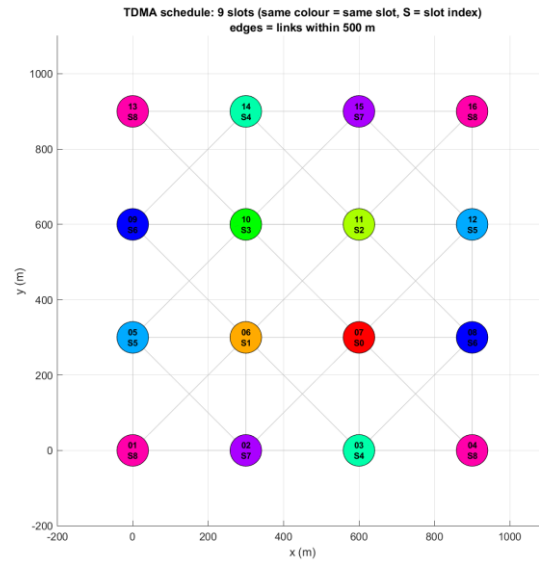


Figure 1. MATLAB output. Same colour means same slot, and S is the slot number.

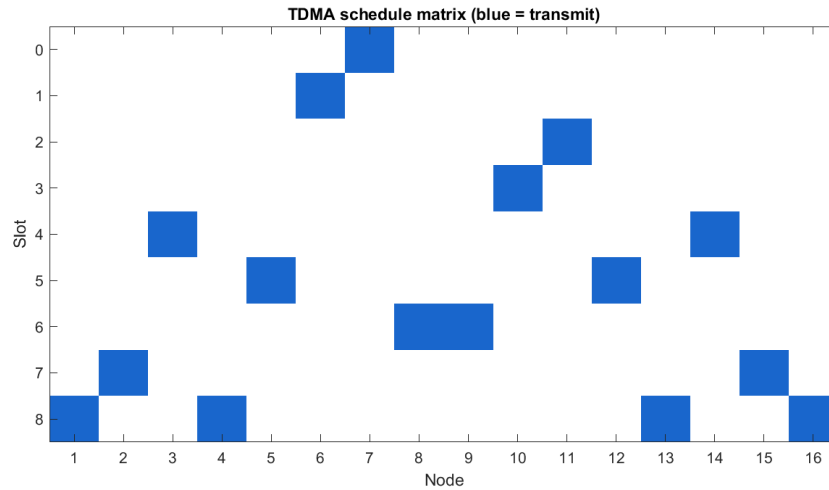


Figure 2. Slot by Node matrix from MATLAB. A filled cell means that radio transmits in that slot.

The MATLAB script also simulates one frame, 84 receptions in all (42 links, both directions) with 1 ms slots. A neighbour receives a packet only if exactly one audible radio is transmitting and the neighbour is not transmitting itself. Table 1 compares my schedule with some deliberately worse ones.

Scenario	Slots	Delivered (of 84)	Delivery	Per second
My schedule (MATLAB)	9	84	100.00%	9333
My schedule (Python)	9	84	100.00%	9333
Wrong: Node_03 in Node_01 slot	9	74	88.10%	8222
Token passing, one radio per slot	16	84	100.00%	5250
All radios in one slot	1	0	0.00%	0
Random slots (mean of 200)	9	41.95	49.94%	4770

Table 1. One-frame packet simulation in MATLAB.

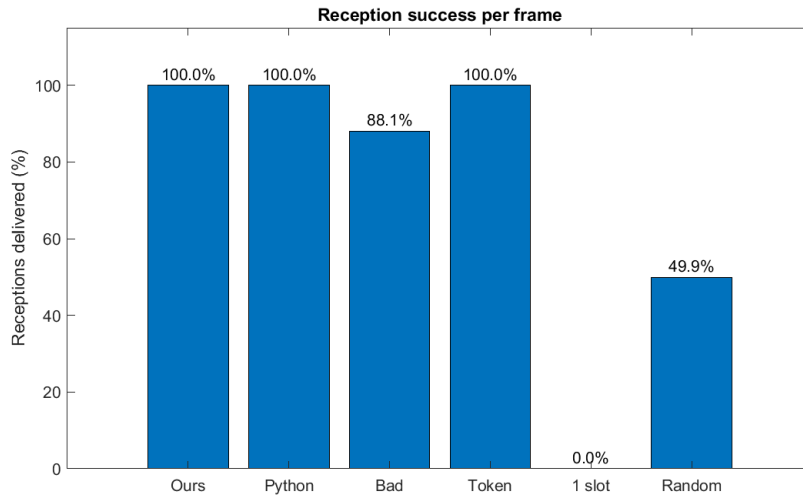


Figure 3. Share of possible receptions delivered in each scenario.

The result is what the theory predicts. My schedule delivers every possible reception in 9 slots. Token passing is also collision-free, but it needs 16 slots, so its throughput is about 1.78 times lower. Moving one radio into the wrong slot already loses 11.9 percent of receptions, and random slots with the same frame length lose about half. So it is the colouring that matters, not just having 9 slots.

MATLAB also repeated the range sweep and a smaller random test. The sweep matches the table above, with one difference: at 650 m MATLAB finds 12 slots but only has a lower bound of 11, so it reports 12 as not proven, while the Python exact search proves it. On 30 random layouts MATLAB averaged 7.63 slots against a mean lower bound of 7.53, matched the bound in 27 of the 30, and gave a mean gain of 2.16 times over one radio per slot. The three it could not prove are cases where MATLAB has no exact search, so they do not necessarily mean a better colouring exists.

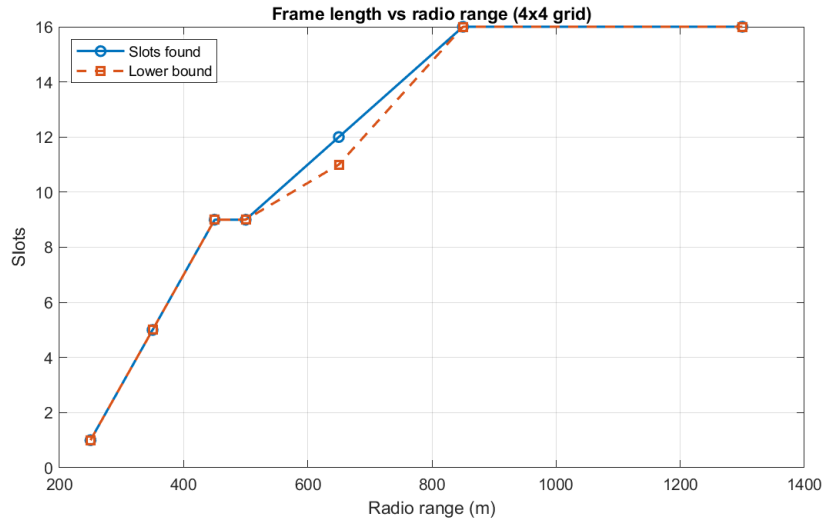


Figure 4. Frame length against radio range on the 4 by 4 grid, from MATLAB.

4.2 A packet-level check in ns-3

To see the schedule work on a separate simulator, I wrote a small ns-3 (version 3.43) program. It uses the 802.11a physical layer only as a raw radio, with a range model that cuts links at 500 m, the same as my graph. TDMA is enforced in the application: each radio sends one broadcast packet exactly at the start of its own slot. I switched off the Wi-Fi random backoff, because otherwise the MAC would quietly hide collisions between radios that share a slot. The program runs the built-in 9-slot schedule and also reads a schedule CSV exported from the Brain.

```
Modules configured to be built:
antenna                      bridge          core
csma                        energy          internet
mobility                    network         propagation
spectrum                   stas            traffic-control
wifi

-- Configuring done (81.8s)
-- Generating done (0.1s)
-- Build files have been written to: /home/susan/tdma/ns-allinone-3.43/ns-3.43/cmake-cache
Finished executing the following commands:
/usr/bin/cmake -S /home/susan/tdma/ns-allinone-3.43/ns-3.43 -B /home/susan/tdma/ns-allinone-3.43/ns-3.43/cmake-cache -DNS3_EXAMPLES=OFF -DNS3_TESTS=OFF -DNS3_ENABLED_MODULES=core,network,i
nternet,mobility,propagation,wifi,csma --warn-uninitialized
[0/2] Re-checking globbed directories...
[10/11] Linking CXX executable /home/susan/tdma/ns-allinone-3.43/build/utills/ns3.43-print-introspected-doxxygen-default
[0/2] Re-checking globbed directories...
[2/3] Linking CXX executable /home/susan/tdma/ns-allinone-3.43/build/scratch/ns3.43-tdma_ns3-default
--- Built-in schedule ---
=== TDMA ns-3 Simulation ===
Nodes: 16 Frame: 9 slots Links: 42 Max RX/frame: 84

=== RESULTS ===
Frames simulated : 10
Expected receptions : 840
Delivered : 840
Packet delivery : 100.00 %
Approx throughput : 18188.6 kbps
Capacity gain vs token passing: 1.78x

--- Python Brain CSV ---
=== TDMA ns-3 Simulation ===
Nodes: 16 Frame: 9 slots Links: 42 Max RX/frame: 84

=== RESULTS ===
Frames simulated : 10
Expected receptions : 840
Delivered : 840
Packet delivery : 100.00 %
Approx throughput : 18188.6 kbps
Capacity gain vs token passing: 1.78x
===
```

Figure 5. ns-3 build and run output for the built-in schedule and the Brain CSV.

Schedule	Frames	Expected	Delivered	Delivery	Gain
Built-in 9-slot schedule	10	840	840	100.00%	1.78x
Brain schedule from CSV	10	840	840	100.00%	1.78x

Table 2. ns-3 results over ten frames.

Both runs report 16 nodes, a 9-slot frame, 42 links and 84 receptions per frame, which matches my graph exactly. Over ten frames that is 840 receptions, and all 840 arrived. The program also printed an approximate throughput of 18108.6 kbps. That number comes from ns-3's own packet size and timing, so I do not compare it with the 9333 receptions per second from MATLAB.

I want to be clear about what this shows. ns-3 here stands in for the radio, not for EMANE. It confirms that my schedule has no collisions when replayed on a different simulator with a real propagation model, which is a stronger check than my own checker alone. I only ran the two collision-free schedules in ns-3, so the wrong, random and single-slot comparisons come from MATLAB, and the EMANE results in section 5 remain the closest test to a real TDMA radio.

5. Part 2: EMANE

The bridge, `emane_bridge.py`, turns the Brain's output into an EMANE schedule file. Node_`NN` becomes NEM number `NN`, a frame has one multiframe, each slot lasts 1 ms, and each slot lists the radios that transmit in it. A radio is in receive mode in every slot not listed for it. The bridge also writes pathloss events so that the emulated radios hear only neighbours within 500 m, matching the graph.

EMANE only ships packages for Ubuntu 22.04 and 24.04 and my laptop runs Ubuntu 26.04 under WSL2, so I used a Docker image based on Ubuntu 24.04 with EMANE 1.5.3. A script builds a small virtual lab inside it. Every radio sits in its own network namespace with its own EMANE process, and a bridge carries the control traffic between them.

For the test I picked Node_`_01` and Node_`_03`. They are 600 m apart, so they cannot hear each other, but both are 300 m from Node_`_02`. That makes them hidden terminals, which is exactly what my schedule is meant to protect against. I ran three phases: my schedule, then a wrong schedule with Node_`_03` moved

into Node_01's slot, then my schedule again. In each phase both radios sent UDP traffic to Node_02 at the same time for 15 seconds.

Phase	01 to 02 loss	03 to 02 loss
My schedule	7.7%	6.4%
Wrong schedule	30%	no result
My schedule again	14%	11%

Every radio I checked accepted each schedule, and the acceptance counter went up from 1 to 2 to 3 with no rejections. Node_01 to Node_02 worked, while Node_01 to Node_16 (1273 m) lost every packet in every run, so the range model does what it should. NEM 1's schedule table shows one transmit slot (slot 8) and eight receive slots, which matches the Brain.

6. What the test does and does not show

The result agrees with the idea behind the colouring. When the two hidden terminals were kept apart they delivered 86 to 94 percent of their packets, and when they were forced to collide the loss was clearly higher. Earlier runs, which had no warm-up, pointed the same way: 12 to 35 percent loss with the good schedule and 32 to 42 percent with the bad one.

I do not want to oversell it. It was one laptop with 1 ms slots and only a few runs, and the loss with the good schedule is not zero because timing on a normal computer is not perfect. I treat it as consistent with the theory, not as a measurement. Ping loss was 40 to 60 percent in every phase, so I only use pings to show reachability. The Node_03 flow in the wrong-schedule phase failed to finish in two of three runs, and I have not found out why, so I make no claim about the cause.

Some problems along the way: the first start failed because the XML used absolute file paths, which I fixed with plain file names. Installing every EMANE package pulled in about 1 GB, so the Dockerfile now installs only what the demo needs. My first test also had no warm-up, which made the numbers noisy, so the final script warms up first.

7. Limits and next steps

- The model treats a link as either on or off. Real radios have fading and capture, and EMANE has packet completion curves that could model that.
- The radios are fixed. If they moved, I would recolour only the affected two-hop area.
- Each radio gets one slot per frame. Traffic between specific pairs would be better served by scheduling links and weighting them by demand.
- Only one frequency is used. EMANE slots can carry different frequencies, which could shorten the frame.
- The EMANE test needs more runs and a quieter machine before it can count as a real measurement.

8. Conclusion

The Brain does what the task asked. It turns 16 coordinates into a conflict-free schedule, reuses slots between radios that are far apart, and says whether the answer is the best possible. For the demo grid that is 9 slots against 16, and it is proven minimal. The independent checker and the 77 tests give me good confidence that no two radios within two hops share a slot.

Part 2 went further than I expected when I started. The schedule loads into EMANE without rejection, the range model behaves as designed, and the packet loss between two hidden terminals was clearly lower with my schedule than with one that made them collide. I am careful about how strongly I word this, because it is a small experiment on one laptop. What it gives me is a working pipeline from coordinates to schedule to emulated packets, and a clear list of what to improve next: a more realistic radio model, mobile nodes, and more repeated runs.

Working on this taught me that a heuristic on its own is not enough to trust, and that a lower bound makes the result much easier to defend. It also taught me to report an experiment as it happened, including the run that failed.

9. References

- [1] A. A. Hagberg, D. A. Schult and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in Proceedings of the 7th Python in Science Conference (SciPy 2008), 2008, pp. 11-15.
- [2] D. Brelaz, "New methods to color the vertices of a graph," Communications of the ACM, vol. 22, no. 4, pp. 251-256, 1979.
- [3] S. Ramanathan and E. L. Lloyd, "Scheduling algorithms for multihop radio networks," IEEE/ACM Transactions on Networking, vol. 1, no. 2, pp. 166-177, 1993.
- [4] Adjacent Link LLC. EMANE (Extendable Mobile Ad-hoc Network Emulator), documentation and TDMA Radio Model. <https://emane.io>

Resume of Sudarshan V:

https://drive.google.com/file/d/1qvv8MSny8AOLjq_RA7kGPKbyn1bnn3J1/view?usp=sharing